



**МОСКОВСКИЙ УНИВЕРСИТЕТ ИМЕНИ С.Ю. ВИТТЕ**

Факультет Информационных технологий  
Кафедра Кафедра информационных систем  
Направление подготовки/Специальность Прикладная информатика

## ОТЧЕТ

о прохождении

Производственная практика

/вид практики/

Эксплуатационная практика

/тип практики/

Студента Чайковской Илоны Александровны  
(фамилия, имя, отчество)

Место прохождения практики ЧОУВО «МУ им. С.Ю. Витте»

Период прохождения практики с 20.04.2026 г. по 31.05.2026 г.

г. Москва 2026 г.

## Содержание

|                                                                   |    |
|-------------------------------------------------------------------|----|
| Введение к практике: Оценка позы человека (Pose Estimation) ..... | 3  |
| Глава 1. АНАЛИЗ ЗАДАНИЯ ЭКСПЛУАТАЦИОННОЙ ПРАКТИКЕ.....            | 7  |
| 1.1: Выбор и обоснования необходимых библиотек и модели.....      | 7  |
| 1.2 Поэтапная декомпозиция задания .....                          | 8  |
| 1.3 Анализ data_set .....                                         | 8  |
| Глава 2. Установка и настройка программного обеспечения .....     | 10 |
| 2.1 Установка библиотек и создание среды.....                     | 10 |
| 2.2 Инициализация модели детекции позы .....                      | 10 |
| 2.3. Детектирование позы на статичных изображениях .....          | 10 |
| Глава 3. Обработка видеопотока в реальном времени .....           | 12 |
| 3.1 Подключение и обработка кадров с веб-камеры .....             | 12 |
| 3.2 Работа в реальном времени .....                               | 12 |
| Глава 4. Математическая обработка координат .....                 | 14 |
| 4.1 Вычисление углов в суставах .....                             | 14 |
| 4.2 Определение позы .....                                        | 14 |
| 4.3 Извлечение координат .....                                    | 15 |
| Вывод по эксплуатационной практике .....                          | 17 |
| СПИСОК ЛИТЕРАТУРЫ.....                                            | 18 |

## **Введение к практике: Оценка позы человека (Pose Estimation)**

### **Зачем это нужно: анализ движений**

Оценка позы (pose estimation) — задача компьютерного зрения, решаемая методами машинного обучения и глубоких нейросетевых архитектур. Современные подходы на основе искусственного интеллекта позволяют анализировать визуальные данные (изображения и видеопоследовательности) для определения пространственного положения и ориентации человека или объекта. Решение задачи базируется на детекции ключевых точек (keypoints), аппроксимирующих кинематическую структуру объекта.

Компьютерное зрение (computer vision) — подобласть искусственного интеллекта, в рамках которой применяются модели машинного обучения (включая сверточные и рекуррентные нейронные сети) для интерпретации визуальной информации на уровне пиксельных и семантических признаков.

Модель машинного обучения представляет собой параметрическую функцию, оптимизируемую для выявления закономерностей в обучающих данных с целью последующего прогнозирования или принятия решений. В рамках данной задачи используется архитектура `pose_landmarker_heavy` из библиотеки MediaPipe, характеризующаяся максимальной точностью среди реализаций семейства. Модель детектирует 33 ключевых точки (от области носа до фаланг пальцев стоп), что значительно превосходит типовую конфигурацию с 17 ключевыми точками, применяемую в ряде альтернативных решений. Выходными данными модели служат нормализованные координаты (значения в диапазоне  $[0,1]$ , выраженные в долях относительно ширины и высоты изображения, а не в абсолютных пикселях) и трёхмерные координаты ( $x, y, z$ ) в метрическом или нормализованном пространстве.

Для оценки качества работы модели вводятся следующие метрики:

Полнота (Recall) — доля истинно положительных ключевых точек, корректно обнаруженных моделью. Критична для минимизации пропусков значимых точек. Точность (Precision) — доля детектированных точек, действительно являющихся корректными ключевыми точками. Необходима для контроля ложных срабатываний. Видимость (Visibility) — показатель, характеризующий присутствие ключевой точки в кадре без учёта окклюзий или выхода за границы изображения. Данная метрика важна для различения «необнаруженной» и «физически отсутствующей/невидимой» точки.

`min_tracking_confidenc` — пороговый параметр уверенности для механизма отслеживания (tracking) между кадрами. Повышение данного порога увеличивает робастность, но приводит к росту задержки обработки (latency) из-за необходимости повторной инициализации детектора. Таким образом, предложенный пайплайн обеспечивает баланс между точностью локализации ключевых точек и вычислительной эффективностью за счёт управления порогами уверенности и использованием нормализованного представления координат.

## ЗАДАЧА 1: Настройка среды и установка библиотек

Подзадачи:

- 1.1 Установить MediaPipe
- 1.2 Установить OpenCV, NumPy, Pandas, Matplotlib
- 1.3 Настроить импорты для нового API (`mediapipe.tasks.python.vision`)

## ЗАДАЧА 2: Инициализация детектора позы

Подзадачи:

- 2.1 Настроить `PoseLandmarkerOptions` с параметрами `confidence`
- 2.2 Указать режим работы `IMAGE` для фото и `VIDEO` для веб-камеры
- 2.3 Создать словарь `LANDMARK_NAMES` для 33 ключевых точек

### ЗАДАЧА 3: Обработка статичных изображений (этапы 1-2)

Подзадачи:

- 3.1 Конвертировать BGR  $\rightarrow$  RGB  $\rightarrow$  mp.Image
- 3.2 Получить detection\_result.pose\_landmarks[0]
- 3.3 Вывести таблицу с координатами
- 3.4 Нарисовать скелет через ручную отрисовку соединений
- 3.5 Подписать номера точек на кадре

### ЗАДАЧА 4: Реализация функции вычисления углов

Подзадачи:

- 4.1 Добавить защиту от деления на ноль (np.clip)
- 4.2 Конвертировать радианы в градусы
- 4.3 Применить для коленей

### ЗАДАЧА 5: Работа с веб-камерой в реальном времени

Подзадачи:

- 5.1 Захват видео через cv2.VideoCapture(0)
- 5.2 Расчёт FPS через разницу времени между кадрами
- 5.3 Конвертация BGR $\rightarrow$ RGB $\rightarrow$ mp.Image на каждом кадре
- 5.4 Отрисовка скелета и углов поверх видеопотока
- 5.5 Вывод FPS на экран

### ЗАДАЧА 6: Реализация счётчика упражнений (приседаний)

Подзадачи:

- 6.1 Создать класс ExerciseCounter с состояниями 'up' и 'down'
- 6.2 Использовать гистерезис (два порога: 90° и 160°) для устранения дребезга
- 6.3 Увеличивать счётчик при переходе 'down'  $\rightarrow$  'up'
- 6.4 Выводить текущий счёт и фазу на кадр

### ЗАДАЧА 7: Анализ и визуализация метрик

Подзадачи:

- 7.1 Сохранять visibility каждой точки в историю (100-200 кадров)
- 7.2 Построить графики изменения  $x$ ,  $y$ ,  $z$ , visibility во времени (Matplotlib)
- 7.3 Реализовать классификатор позы STANDING/SITTING по расстоянию плечи-бёдра (порог 0.35)
- 7.4 Реализовать проверку симметрии

## **Глава 1. АНАЛИЗ ЗАДАНИЯ ЭКСПЛУАТАЦИОННОЙ ПРАКТИКЕ**

Необходимо разработать программу, которая будет находить позы человека через ключевые точки. Human Pose Estimation (HPE) — это способ идентификации и классификации узлов человеческого тела. Это способ определения координат каждого узла (руки, головы, туловища и так далее), называемого ключевой точкой и определяющего положение тела человека. Связь между этими точками называется парой. Образованное между точками соединение должно быть существенным, то есть не все точки могут образовывать пару. Изначально задача HPE заключается в формировании скелета описания человеческого тела с последующей обработкой для конкретных задач.

### **1.1: Выбор и обоснования необходимых библиотек и модели**

Для создания программы оценка позы человека необходимо использовать готовые библиотеки и модель. Модель уже была определена по заданию. MediaPipe — это модель от Google, которая состоит из множества натренированных моделей. MediaPipe Model Maker — это инструмент для настройки существующих моделей машинного обучения (ML) для работы с вашими данными и приложениями. MediaPipe работает как программная платформа для обработки потоков данных. Данные перемещаются через ряд соединенных между собой «вычислителей», каждый из которых выполняет определенную задачу над данными, прежде чем передать их следующему. На приведенном выше графике эти вычислители показаны в виде узлов, соединенных потоками данных. Каждый поток представляет собой последовательность пакетов данных. Такая структура графика упрощает пошаговую обработку данных и обеспечивает эффективную работу конвейера машинного обучения. Конвейеры обработки данных, созданные с помощью MediaPipe, могут бесперебойно работать в веб-приложениях, на смартфонах (Android и iOS) и даже в небольших встроенных системах. Его

кроссплатформенные возможности помогают разработчикам создавать захватывающие и отзывчивые приложения для любых устройств.

Библиотека OpenCV нужна для того, чтобы читать и записывать изображения, захват видео, обрисовывать точки и линии, а также вывод изображений на экран. Numpy необходим для вычисления углов через скалярные произведения. Pandas сохраняет точки в структурированном формате и экспортирует данные в CSV-файл. Matplotlib библиотека, которая позволяет построить графики и сохранить их в PNG формате.

## **1.2 Поэтапная декомпозиция задания**

Первым делом мы устанавливаем модель и необходимые библиотеки. Также разработана функция для тестового изображения, которая показала таблицу с координатами, это было сделано чтобы понять корректно ли работает программа. Далее, детектируем изображение, а после обрабатываем 5 фотографий людей в разных позах. Все пять изображений были обработаны и был отрисован скелет человека. После подключаем камеру и каждый кадр обрабатываем детектором позы. Также каждый кадр обрабатывается и конвертируется в RGB, только такой формат воспринимает библиотека MediaPipe. Для вычисления угла разработана функция `calc_angle`, она то и принимает три точки и после возвращает угол градусов в средней точке. Вычисляется по формуле косинуса угла.

## **1.3 Анализ data\_set**

Тест модели состоял из 100 фотографий людей в самых разных позах и с разной внешностью. Оказалось, что лучше всего модель справляется, когда человек стоит в полный рост, его ноги и руки видны целиком, и ничего не перекрывает. В таких случаях мы уверены в положении всех 33 точек на 80% и выше. Когда человек сидит, модель работает уже не так хорошо, особенно с коленями и ступнями – они могут быть видны только на 40-60%, потому что их частично закрывает тело. Самые большие проблемы возникают, когда

человек сильно перекрывает сам себя, например, скрестив руки. Тогда некоторые точки, как запястье, видны меньше чем на 30%, и модель начинает их "придумывать" в неправильных местах. Спортивные позы, вроде глубоких приседаний, хорошо определяются для верхней части тела, но ступни иногда теряются из-за того, как снято фото. Поэтому, чтобы программа работала стабильно в реальном времени, важно, чтобы было хорошее освещение, а камеру нужно ставить так, чтобы человек помещался в кадр целиком, и его руки-ноги не закрывали друг друга.

## Глава 2. Установка и настройка программного обеспечения

### 2.1 Установка библиотек и создание среды

Для начала создали виртуальную среду `.venv`. Для это открыли терминал и вписали такую команду `python3 -m venv` и туда же установили библиотеку MediaPipe и все последующие библиотеки с помощью команды через терминал `pip install mediapipe opencv-python numpy pandas matplotlib`. После того, как была готова половина от задания, то есть тестовый запуск программы для определения точек на изображении, программа выдала ошибку. Ошибка заключалась в том, что он не находил модуль `solutions`, который должен был входить автоматически в MediaPipe. Оказалось, что в новой версии MediaPipe удалили данный модуль. Чтобы решить эту ошибку, пришлось перейти на новый API Tasks, но и это не совсем помогло, необходимо было скачать модель напрямую только после этого перестало выдавать ошибку.

### 2.2 Инициализация модели детекции позы

Сначала написали функцию, которая автоматически загружала предобучение модели из официального репозитория MediaPipe. Потом создали базовые настройки и указали путь к модели. Настроили параметры детектора с помощью `PoseLandmarkerOptions`, а именно указали режим выполнения, задали максимальное число обнаружений и поставили пороговые значения. После уже детектор создавался через метод `create_from_options` и только потом было можно обрабатывать изображения и видеокадры.

### 2.3. Детектирование позы на статичных изображениях

Детектирование поз человека на статичных изображениях указывали путь к самому файлу с изображением, а потом загружали его с помощью библиотеки OpenCV, программа конвертировала из BGR в RGB, потому что модель MediaPipe только такой формат и принимает. RGB преобразуется в специальный объект `mp.Image`, так как он является универсальным форматом для изображений. Потом изображения передавались в метод `detect`, он

возвращает результат, который содержит список точек. Если же все прошло успешно, и он определил точки, то результат извлекается из массива. После того, как получили координаты, то на изображении ставились ключевые точки и в последующем линии, которые соединяли точки. Если же изображение имеет плохое качество, то точки будут отображаться красным цветом, это сделано для того, чтобы мы могли оценить качество детекции.

## Глава 3. Обработка видеопотока в реальном времени

### 3.1 Подключение и обработка кадров с веб-камеры

В рамках функции `main_webcam()` реализован механизм для работы с видеопотоком от веб-камеры в режиме реального времени. Первым шагом является настройка захвата видео с использованием `cv2.VideoCapture(0)`, где число 0 указывает на стандартную, подключенную к компьютеру веб-камеру. После успешного установления соединения с камерой, программа переходит в бесконечный цикл. На каждой итерации этого цикла происходит считывание нового кадра с помощью метода `cap.read()`. Этот метод возвращает два значения: индикатор успешности операции и само изображение. Поскольку модель MediaPipe требует входные данные в цветовом пространстве RGB, каждый полученный кадр (изначально в формате BGR) конвертируется с помощью `cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)`. Далее, это RGB-изображение преобразуется в специальный объект `mp.Image`, который является стандартным форматом для нового API MediaPipe Tasks. Подготовленный кадр затем передается детектору позы через метод `detect`. Если в кадре обнаружен человек, детектор возвращает координаты всех 33 ключевых точек его тела. На основе этих координат далее производятся расчеты: вычисляются углы в суставах, обновляется счетчик выполненных упражнений, проводится анализ симметрии движений и определяется, стоит человек или сидит. После завершения всех вычислений, на исходный BGR-кадр наносятся различные визуальные элементы: скелет человека (представленный кружками и линиями), текстовые данные о текущем количестве кадров в секунду (FPS), значения углов в суставах, счетчик.

### 3.2 Работа в реальном времени

Реализация визуализации скелета в реальном времени осуществляется в рамках основного цикла обработки кадров веб-камеры, после получения 33 ключевых точек от детектора MediaPipe. Поскольку новый Tasks API не

поддерживает утилиту `mp.solutions.drawing_utils` из предыдущих версий, отрисовка скелета была реализована вручную с использованием функционала `OpenCV`. Процесс включает вычисление пиксельных координат для каждой из 33 точек путём масштабирования нормализованных значений  $x$  и  $y$  на ширину и высоту текущего кадра. Затем, на изображении отображаются зелёные кружки радиусом 3 пикселя. Для оценки надёжности детекции, точки с атрибутом `visibility` ниже 0.5 визуализируются красным цветом. Далее, на кадр наносятся линии, соединяющие точки согласно анатомическим связям (например, пары плечо-локоть-запястье, бедро-колени-ступня). Эти соединения отрисовываются зелёными линиями толщиной 2 пикселя, создавая эффект наложенного скелета. Дополнительно, для повышения наглядности, на суставах отображаются подписи с текущими значениями углов в градусах. Такая визуализация позволяет в реальном времени отслеживать динамику позы, оценивать точность работы модели и осуществлять визуальный контроль за корректностью выполнения упражнений. Цветовое кодирование точек (зелёный – высокая надёжность, красный – низкая надёжность) обеспечивает мгновенную обратную связь о качестве детекции, способствуя корректировке положения пользователя или улучшению условий освещения для повышения точности модели.

## Глава 4. Математическая обработка координат

### 4.1 Вычисление углов в суставах

Расчёт углов в суставах является фундаментальным математическим инструментом для изучения биомеханики движений человека. Эта операция инкапсулирована в функцию `calc_angle(A, B, C)`, которая принимает три точки в формате (x, y) и возвращает величину угла в градусах, вершиной которого является средняя точка B. Алгоритм базируется на векторной алгебре: сначала создаются векторы BA (от B к A) и BC (от B к C), затем вычисляются их скалярное произведение и нормы (длины). Косинус искомого угла определяется как частное от деления скалярного произведения на произведение норм векторов. Далее, путём применения арккосинуса и преобразования из радиана в градусы (умножением на  $180/\pi$ ), получается конечный угол. Для обеспечения устойчивости к ошибкам округления, значение косинуса перед вычислением арккосинуса нормализуется в пределах  $[-1, 1]$  с помощью функции `pr.clip`. В контексте программы, данная функция применяется для анализа четырёх суставов: левого и правого локтей (используя точки плечо-локоть-запястье) и левого и правого коленей (используя точки бедро-колени-ступня). Результаты вычислений визуализируются на видео, используются для определения фаз движения в счётчике приседаний и для оценки симметрии движений путём сопоставления углов в парных суставах.

### 4.2 Определение позы

На основе вычисленных углов в суставах программа рассчитывает ряд биомеханических параметров, позволяющих оценить качество движений человека и выявить возможные нарушения. К таким параметрам относятся средний угол сгибания коленей, который вычисляется как арифметическое среднее между углами левого и правого колена, и используется в счётчике приседаний для определения текущей фазы движения (опускание или подъём).

Также проводится анализ разницы между углами левого и правого колена и левого и правого локтя — если же разница выходит за пороговое значение в 15 градусов, то программа выводит предупреждение о том, что поза несимметрична, а это важно учитывать для физиотерапии и контроля правильности выполнения упражнений. Для классификации позы стоя или сидя вычисляется вертикальное расстояние между средними координатами плеч и бёдер по оси Y (в нормализованных значениях от 0 до 1), если это расстояние выходит за 0.35, то делается вывод о том, что человек находится в положении стоя, в противном случае — сидя. Дополнительно анализируется относительная глубина (координата z) для каждой точки, где отрицательные значения указывают на приближение части тела к камере, а положительные — на удаление, что позволяет оценивать движения в трёхмерном пространстве, хотя в данной работе этот параметр используется в основном для информационных целей. Все рассчитанные биомеханические параметры в реальном времени выводятся на видеокадр, предоставляя пользователю наглядную обратную связь о его движениях и осанке.

### 4.3 Извлечение координат

В программе, есть отдельная функция, которая предназначена для сбора статистических данных о движениях человека и последующей визуализации изменений позы во времени, реализованы функции анализа временных рядов и записи координат. Во время обработки видеопотока для каждого кадра фиксируются нормализованные координаты x, y, z и показатель видимости (visibility) восьми ключевых точек: левого и правого плеча, локтей, коленей и ступней. Данные сохраняются в циклический буфер, который накапливает информацию до 200 кадров, после чего автоматически создаются графики и формируется CSV-файл. Для каждой ключевой точки формируется словарь, где в списках хранятся её координаты с привязкой к номеру кадра. По окончании сбора данных вызывается функция `plot_coordinates_history`, строящая четыре подграфика — изменения координат X, Y, Z (глубина) и

метрики видимости во времени, отражающей надёжность обнаружения точки на протяжении всего периода записи. Одновременно функция `save_coordinates_csv` экспортирует все данные в CSV-файл, где каждая строка содержит номер кадра, название точки и её координаты с показателем видимости. Такой подход позволяет не только визуально отслеживать динамику движений через графики, но и выполнять дальнейший статистический анализ в сторонних программах, например, в Excel или специализированных научных пакетах. Анализ временных рядов особенно полезен для выявления закономерностей движений, оценки стабильности выполнения упражнений и обнаружения отклонений в позе на протяжении длительного времени.

### **Вывод по эксплуатационной практике**

В ходе выполнения задания по эксплуатационной практике были сделаны выводы. Во-первых, оказалось, что Pose Estimation возвращает не «силуэт», а конкретные координаты ключевых 33 точки, которые были представлены в значениях  $x$ ,  $y$ ,  $z$ ,  $visibility$ , где  $x$ ,  $y$  – нормализованные координаты, а  $z$ - относительная глубина (насколько близко/далеко находится объект), а  $visibility$  показывает надёжность нахождения каждой точки (от 0 до 1). Во-вторых, поняли как из координат получить геометрические признаки: углы, расстояния, симметрию. Для этого реализовали вычисления через формулу косинуса угла между векторами, произвели расчет расстояния между плечами и бедрами для классификации позы и сравнили анализ симметрии с помощью вычислений между парными суставами. В-третьих, реализовали счётчик повторений упражнений, где был применён принцип машины состояний (state machine) с двумя фазами 'up' и 'down', а ключевым элементом стал гистерезис — это использование двух разных порогов угла ( $90^\circ$  для перехода в фазу 'down' и  $160^\circ$  для возврата в 'up'), что помогло повысить устойчивость к шумам, то есть срабатывания при колебаниях угла вокруг граничного значения- считались ложными. Готовая модель MediaPipe Pose является законченным инструментом, который не требует обучения или тонкой настройки, а основная задача программиста — это создание «логики поверх неё», то есть извлечение, интерпретация и применение выходных данных модели для решения конкретных прикладных задач — от спортивной аналитики до физиотерапии и контроля осанки.

## СПИСОК ЛИТЕРАТУРЫ

1. Google [Электронный ресурс]. – URL: [https://ai.google.dev/edge/mediapipe/solutions/model\\_maker?hl=ru](https://ai.google.dev/edge/mediapipe/solutions/model_maker?hl=ru) (дата обращения: 25.07.2025)
2. Коллектив, авторов Нейросеть. Пошаговое руководство по генерации картинок и текста / авторов Коллектив. – Москва: АСТ, 2024. – 128 с. – ISBN 978-5-17-158109-1.
3. McKinney W Python for Data Analysis: Data Wrangling with Pandas, NumPy, and IPython. – English: O'Reilly Media, 2017. – 550 p.
4. Беляков, А. Практическое применение библиотек SciPy и NumPy / А. Беляков. – СПб : Питер, 2020. – 250 с.
5. Машинное обучение без кодинга для фронтенд-задач. Разбираем работу фреймворка MediaPipe [Электронный ресурс]. – URL: <https://habr.com/ru/companies/oleg-bunin/articles/735024/> (дата обращения: 01.10.2024).